

Fundamentos de Teste e Qualidade de Software

Da intenção ao resultado observável



Prof. Dr. João Choma

UEM

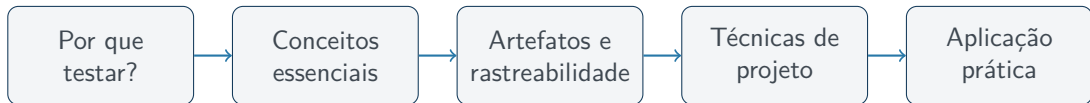
github.com/JoaoChoma

Fundamentos



Ao final, você deverá ser capaz de:

- explicar por que testamos e reconhecer os limites do teste;
- diferenciar verificação, validação, engano, defeito, erro e falha;
- relacionar requisito, risco, caso, execução e evidência;
- reconhecer os principais artefatos de teste;
- projetar casos com equivalência e valores-limite.





Ideia central

Teste de software é uma investigação planejada para obter informação sobre a qualidade e revelar diferenças entre o esperado e o observado.

- Executamos o software em condições escolhidas.
- Comparamos resultados com oráculos ou critérios esperados.
- Produzimos evidências para apoiar decisões.

Atenção

Testar não é apenas “usar o sistema”: existe objetivo, método e resultado esperado.



Perspectiva	Pergunta orientadora
Correção funcional	Faz o que foi especificado?
Adequação ao uso	Resolve a necessidade real?
Confiabilidade	Mantém o comportamento nas condições previstas?
Segurança	Protege dados e operações?
Usabilidade	Pode ser usado com eficácia e clareza?
Desempenho	Responde no tempo e com os recursos esperados?

Teste fornece evidências sobre qualidade; não cria qualidade sozinho.



Perspectiva	Teste de uma transferência PIX
Correção	R\$ 100 debitados e creditados exatamente
Adequação	usuário encontra e conclui a operação
Confiabilidade	queda de conexão não duplica a transferência
Segurança	outro usuário não acessa o comprovante
Usabilidade	erros explicam como corrigir os dados
Desempenho	confirmação ocorre no tempo acordado

Um fluxo pode “funcionar” e ainda falhar em segurança, clareza ou desempenho.



Verificação

Construímos corretamente?

- Produto versus especificação.
- Revisões e análise estática.
- Testes técnicos.

Validação

Construímos a coisa certa?

- Solução versus necessidade.
- Testes de aceitação.
- Uso representativo.

Um produto pode ser fiel a uma especificação inadequada. Precisamos das duas perspectivas.



Requisito: “permitir agendamento de segunda a sexta”.

Verificação

- Revisar: feriados contam?
- Bloquear sábado e domingo.
- Conferir todos os critérios.

Validação

A recepcionista informa que a clínica também atende aos sábados.

O sistema pode cumprir o requisito e não atender à necessidade real.



Demonstrar conformidade

- Evidenciar requisitos atendidos.
- Apoiar aceite e liberação.

Encontrar problemas

- Escolher entradas reveladoras.
- Reduzir incerteza nos riscos.

Propósito explícito

Um teste útil procura confirmar uma expectativa ou tenta refutá-la.



Testar amostras não demonstra correção para todas as entradas, estados, ambientes e sequências possíveis.

Edsger W. Dijkstra, 1972

“Testes podem mostrar a presença de bugs, mas nunca mostrar sua ausência.” (tradução livre)

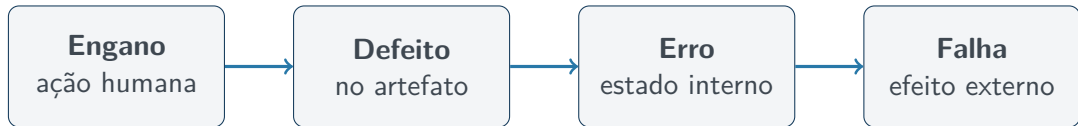
Consequência: priorizar por risco, combinar técnicas e registrar o que foi — e o que não foi — coberto.



Funcionalidade	Prob.	Impacto	Prioridade
Cobrança duplicada	média	muito alto	máxima
Cupom incorreto	alta	médio	alta
Foto desalinhada	alta	baixo	menor

Primeiro testamos pagamento, idempotência e recuperação. A ordem do trabalho segue o risco, não apenas a facilidade do teste.

Linguagem comum



Nem todo defeito é ativado; nem todo erro interno se propaga até uma saída percebida pelo usuário.



Regra: clientes com 10 ou mais itens recebem 10% de desconto.

Implementação

```
se quantidade > 10: aplicar desconto
```

- 1 O desenvolvedor interpreta mal a fronteira (**engano**).
- 2 Escreve `> 10` em vez de `>= 10` (**defeito**).
- 3 Com 10 itens, o desconto permanece zero (**erro**).
- 4 O total exibido é maior que o esperado (**falha**).

Com 12 itens, o mesmo defeito não falha: a entrada importa.



Teste	Depuração
Revela uma falha	Localiza e corrige a causa
Compara esperado e observado	Investiga estado, fluxo e código
Produce evidência	Produce uma alteração
Pode ignorar a implementação	Geralmente exige conhecimento interno

Após a correção, reexecutamos o teste que falhou e testes de **regressão**.



Falha: ao alterar a quantidade no carrinho, o frete desaparece.

- ① **Teste:** reproduz e registra o problema.
- ② **Depuração:** localiza a substituição do objeto de frete.
- ③ **Correção:** preserva a modalidade selecionada.
- ④ **Confirmação:** reexecuta o caso que falhou.
- ⑤ **Regressão:** verifica cupom, subtotal, frete grátis e endereço.

Risco

Corrigir o sintoma sem testar efeitos colaterais pode introduzir outra falha.



Nível	Foco	Exemplo
Unidade	componente isolado	função de desconto
Integração	interfaces	serviço + banco
Sistema	produto completo	compra ponta a ponta
Aceitação	necessidade do negócio	usuário confirma o fluxo

Os níveis são complementares. Encontrar tudo apenas no teste de sistema custa mais.



Nível	Teste concreto
Unidade	<code>calcularTotal(100, 10%)</code> devolve 90
Integração	serviço grava itens e consulta o estoque
Sistema	cliente adiciona produto, paga e recebe confirmação
Aceitação	negócio confirma parcelamento e cancelamento

Uma falha no cálculo pode ser descoberta antes de uma compra ponta a ponta.



Caixa-preta requisitos, entradas e saídas, sem depender do código.

Caixa-branca decisões, caminhos, condições e fluxo interno.

Experiência exploração, erros recorrentes e conhecimento do domínio.

Escolha orientada ao risco

Uma estratégia madura combina as três conforme impacto, probabilidade e contexto.



Exemplo: três estratégias no login

Caixa-preta credenciais válidas, senha errada, vazios e bloqueado.

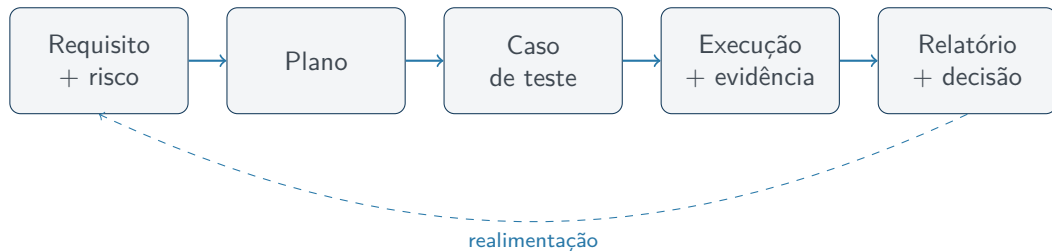
Caixa-branca ramos “inexistente”, “senha inválida”, “bloqueado” e “autenticado”.

Experiência espaços no e-mail, maiúsculas, várias tentativas, botão voltar e sessão expirada.

Resultado

As abordagens encontram classes diferentes de problema no mesmo recurso.

Artefatos e rastreabilidade



Artefato

Entrega criada ou atualizada para comunicar planejamento, projeto, execução ou resultado.



Descreve a abordagem geral:

- objetivos, escopo e itens fora de escopo;
- riscos, prioridades, níveis, tipos e técnicas;
- ambiente, dados, recursos e responsabilidades;
- cronograma e critérios de entrada, saída e suspensão;
- entregáveis e comunicação dos resultados.

Pergunta respondida

O que, por que, como, quando e por quem testar?



Elemento	Cadastro de pacientes — versão 1.3
Objetivo	avaliar cadastro, edição e busca
Fora do escopo	importação da versão antiga
Maior risco	paciente duplicado ou CPF associado incorretamente
Estratégia	API, interface, equivalência e limites
Ambiente	homologação com banco anonimizado
Entrada	build implantada e estável
Saída	riscos críticos testados; sem falha bloqueadora

O plano é curto, mas torna as decisões explícitas.



Campo	Exemplo
ID e título	CT-IDADE-01 — cadastro com idade mínima
Requisito	RN-07 — idade entre 18 e 65 anos
Pré-condições	tela aberta; usuário não cadastrado
Dados	idade = 18
Passos	preencher campos e confirmar
Resultado esperado	cadastro criado
Prioridade	alta

O caso descreve a intenção; resultado real e status pertencem à execução.



CT-IDADE-02 — aceitar idade mínima

Requisito RN-07 — idade entre 18 e 65, inclusive.

Pré-condição formulário aberto; CPF não cadastrado.

Dados nome Ana; CPF válido; idade 18.

Passos preencher os campos e selecionar “Cadastrar”.

Esperado paciente criado, ID exibido e idade 18 gravada.

Decisão objetiva

O esperado específico permite determinar se o caso passou.



Roteiro sequência operacional para executar um ou mais casos.

Execução versão, ambiente, executor, data, observado e status.

Evidência log, captura ou resposta que sustenta a conclusão.

Relatório cobertura, aprovações, falhas, bloqueios e riscos residuais.

Reuso

Separar especificação e execução permite aplicar o mesmo caso em várias versões e ambientes.



Campo	Registro
Caso	CT-IDADE-02
Build/ambiente	1.3.42 / Chrome 126 / homologação
Executor/data	Maria / 28-07-2026
Observado	ID 981 criado; idade gravada como 18
Status	aprovado
Evidência	resposta HTTP 201 + consulta do registro

Uma captura sem versão, dados ou vínculo com o caso é evidência fraca.



Requisito	Risco	Casos	Execução	Situação
RN-07 Idade	cadastro indevido	CT-01-06	build 42	5 passaram; 1 falhou
RN-12 Login	acesso indevido	CT-07-11	build 42	todos passaram

- Há requisito importante sem teste?
- Qual risco é afetado pela falha?
- O que reexecutar após uma mudança?

Limite

Cobertura indica relações; não prova que os testes sejam bons.



Mudança: idade máxima passa de 65 para 70 anos.

- requisito RN-07 e regra de validação afetados;
- casos CT-04, CT-05 e CT-06 usam o limite antigo;
- novos limites: 69, 70 e 71;
- telas, API e mensagens exibem a regra;
- evidências anteriores não comprovam a nova versão.

Valor da rastreabilidade

A análise de impacto deixa de depender apenas da memória da equipe.



Registre informação suficiente para análise:

- título e referência ao caso ou requisito;
- versão, ambiente e pré-condições;
- passos mínimos para reproduzir;
- resultado esperado e observado;
- evidências;
- severidade e prioridade.

Não são sinônimos

Severidade mede o impacto; **prioridade** indica a urgência de correção.

Exemplo: severidade não é prioridade



Situação	Sever.	Prior.	Motivo
Perda de dados em tela interna	crítica	alta	impacto grave
Logotipo errado na campanha de amanhã	baixa	urgente	prazo comercial
Botão secundário desalinhado	baixa	baixa	sem bloqueio

Os critérios pertencem ao projeto, não ao humor de quem registra.

Técnicas de projeto



Regra

O cadastro aceita idade inteira entre 18 e 65 anos, inclusive.

Testar todas as idades e entradas inválidas é inviável.

- 1 Agrupar entradas que devem receber o mesmo tratamento.
- 2 Escolher representantes desses grupos.
- 3 Intensificar o teste nas fronteiras.



Classe	Intervalo	Tipo	Representante
CE1	$\text{idade} < 18$	inválida	10
CE2	$18 \leq \text{idade} \leq 65$	válida	30
CE3	$\text{idade} > 65$	inválida	70

Também existem classes de tipo e formato: vazio, texto, decimal e inteiro.

Hipótese

Valores da mesma classe devem receber tratamento equivalente.



Regra: e-mail obrigatório no formato local@domínio.

Classe	Tipo	Representante
formato aceito	válida	ana@exemplo.com
ausente	inválida	vazio
sem @	inválida	ana.exemplo.com
sem parte local	inválida	@exemplo.com
sem domínio	inválida	ana@

As classes nascem da regra e do tratamento esperado.



Defeitos aparecem com frequência nas bordas:

17		18		19		64		65		66
fora		mínimo		dentro		dentro		máximo		fora

Combinação

Equivalência reduz o domínio; valor-limite intensifica a investigação nas bordas.



Regra: de 8 a 20 caracteres.

Tamanho	Posição	Esperado
7	mínimo -1	rejeitar
8	mínimo	aceitar
9	mínimo +1	aceitar
19	máximo -1	aceitar
20	máximo	aceitar
21	máximo +1	rejeitar

Composição e caracteres permitidos exigem outras partições.



ID	Entrada	Técnica	Esperado
CT-01	17	limite inferior -1	rejeitar
CT-02	18	limite inferior	aceitar
CT-03	19	limite inferior $+1$	aceitar
CT-04	64	limite superior -1	aceitar
CT-05	65	limite superior	aceitar
CT-06	66	limite superior $+1$	rejeitar
CT-07	vazio	classe inválida	informar obrigatoriedade
CT-08	texto	classe inválida	informar formato inválido

Cada resultado esperado deve ser observável e específico.



Caso fraco

Passo: testar idade inválida.

Esperado: funcionar corretamente.

Caso melhorado

Dado: idade 17; demais válidos.

Ação: selecionar “Cadastrar”.

Esperado: não criar; exibir a faixa 18–65 no campo; preservar os dados.

Clareza transforma uma ideia em verificação reproduzível.



- usar somente um valor “feliz”;
- esquecer classes inválidas e campos vazios;
- escrever “funcionar corretamente” como resultado;
- misturar vários objetivos em um caso;
- copiar passos sem atualizar a regra;
- aprovar sem registrar versão, ambiente ou evidência.

Prática e encerramento



Use os modelos da pasta docs/.

- 1 Escolha uma regra com intervalo ou restrição de formato.
- 2 Registre escopo, risco, ambiente e critério de saída.
- 3 Modele classes válidas e inválidas.
- 4 Crie casos para representantes e valores-limite.
- 5 Troque os casos com outra equipe para revisão.

Entregáveis

Plano curto, tabela de partições e ao menos seis casos.



Checklist de revisão

- Cada caso aponta para um requisito ou risco?
- Pré-condições e dados são reproduzíveis?
- Passos são necessários e suficientes?
- O resultado esperado é observável?
- Válidos, inválidos e limites estão contemplados?
- Dependências estão explícitas?
- A cobertura mais importante está alinhada ao risco?



- ① Teste produz **informação e evidência**.
- ② A cadeia é **engano** → **defeito** → **erro** → **falha**.
- ③ Artefatos conectam planejamento, projeto e execução.
- ④ Rastreabilidade liga requisitos, riscos, testes e resultados.
- ⑤ Equivalência e limites selecionam dados reveladores.



- DIJKSTRA, E. W. *The Humble Programmer*. CACM, 1972.
- ISO/IEC/IEEE 29119-1. *Software Testing — Part 1*.
- PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de Software*.
- SOMMERVILLE, I. *Engenharia de Software*. 10. ed.
- MYERS, G. J.; SANDLER, C.; BADGETT, T. *The Art of Software Testing*.

Perguntas?

Prof. Dr. João Choma

github.com/JoaoChoma